

Computer Science and Engineering Department, SVNIT, Surat
M.Tech. I – Semester I
Design and Analysis of Algorithm (CSDS103)
Lab Assignment 1

General Instructions

1. Develop the algorithm/pseudocode before implementing the program.
2. Do not use built-in functions that directly solve the main problem.
3. Handle appropriate boundary and special cases.
4. Submit the algorithm, program, test cases, and time and space complexity analysis.

Problem 1 – Employee Directory

An organisation maintains the names of n employees in an unsorted list. Design an algorithm to arrange the names in **lexicographical order** and display all names that occur more than once. If no duplicate names exist, report the same.

- (a) Implement the solution without using a built-in sorting function.
- (b) Clearly define how uppercase and lowercase characters are treated during comparison.
- (c) Analyse the best-case, average-case, and worst-case time complexity and auxiliary space complexity of your approach.
- (d) Discuss whether duplicate detection can be performed without sorting and compare the resulting complexity.

Problem 2 – Locally Balanced Matrix

An $n \times n$ integer matrix is said to be **locally balanced** if each element is equal to the sum of its valid neighbouring elements in the four directions: up, down, left, and right. Only neighbours that exist within the matrix are considered.

For each element $A[i][j]$, check whether

$$A[i][j] = A[i-1][j] + A[i+1][j] + A[i][j-1] + A[i][j+1]$$

using only valid neighbours. If the matrix is not balanced, report the first violating element in row-major order along with its position, value, and neighbour sum.

Analyse the time and auxiliary space complexity of your solution and explain how corner, boundary, and interior elements are handled.

Problem 3 – First Unique Employee

A company records the sequence in which employees access a system. An employee is called **unique** if their name occurs exactly once in the complete sequence.

For example,

Amit, Neha, Amit, Rahul, Neha, Karan, Rahul

has **Karan** as the first unique employee.

Given a sequence of n employee names, design two solutions to find the first unique name. The first solution should use only the given list and repeated comparisons, while the second may use an additional data structure.

Compare the two approaches in terms of time complexity, auxiliary space, and suitability for large input.

Problem 4 – Stable Matrix

You are given an $n \times n$ integer matrix. A matrix is called **stable** if every non-boundary element satisfies

$$A[i][j] = A[i-1][j] + A[i+1][j] + A[i][j-1] + A[i][j+1].$$

Determine whether the matrix is stable. If it is not, report the number and positions of the violating elements and the position having the maximum absolute difference between the element and the sum of its neighbours.

$$D(i, j) = |A[i][j] - (A[i-1][j] + A[i+1][j] + A[i][j-1] + A[i][j+1])|$$

Analyse the best-case and worst-case time complexity and auxiliary space complexity. Also state whether an additional matrix is required.

Problem 5 – Nearly Ordered Employee Directory

An employee directory containing n names is expected to be in lexicographical order, but due to a data-entry error, some names may have been placed incorrectly.

Given the list, determine whether it is already sorted, identify the positions where the ordering is violated, and determine whether exchanging only two names can make the complete list sorted. If possible, report the two positions.

For example,

Amit, Karan, Neha, Rahul, Priya

is already sorted, whereas

Amit, Neha, Karan, Rahul, Priya

contains an ordering violation.

Do not sort the complete array merely to determine whether it is already ordered. Analyse the time and auxiliary space complexity of your approach and state the conditions under which a single swap can restore the ordering.

Problem 6 – Dominant Element

Given an integer array of size n , determine whether there exists an element that occurs **more than half of the time**. For example,

2, 2, 1, 2, 3, 2, 2

contains a dominant element because 2 occurs more than $n/2$ times.

Develop two different solutions: one based on repeated counting/comparison and another that reduces repeated work. Compare their time and space complexities and determine whether the dominant element can be identified without using additional storage.

Submission

Submit the algorithm/pseudocode, program, representative test cases including boundary cases, and the time and auxiliary space complexity of the solution. For problems requiring multiple approaches, provide a brief comparison and justify the more efficient approach.